

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Customer Segmentation and Churn Prediction Using the Online Retail II Dataset

Digital Assignment 3

Submitted by

Suyash Agrawaal (Reg No. – 24BDS1167)

Under the Guidance of

Course Faculty: Dr. Meignanamoorthi D

Course: Programming for Data Science (BCSE207L)

Class Number: CH2026270100997

Slot: G1

Semester: Fall Semester 2026-2027

School of Computer Science and Engineering
Vellore Institute of Technology, Chennai

INDEX

Abstract **3**
1. Introduction **4**
1.1 Background 4
1.2 Objectives of This Phase 4
1.3 Tools and Environment 4
2. Final Performance Comparison and Statistical Analysis **5**
2.1 Experimental Setup 5
2.2 Consolidated Model Comparison 5
2.3 Cross-Validation Results 6
2.4 Confidence Intervals for Mean ROC-AUC 7
2.5 Paired Significance Tests Across Folds 8
2.6 Pairwise McNemar Tests on the Test Set 9
2.7 Bootstrap Confidence Intervals 10
2.8 Reconciling the Two Testing Approaches 10
2.9 Final Model Selection 11
3. Consolidated Customer Scoring View **12**
3.1 Combining Risk, Value and Segment 12
3.2 Recommended Action Matrix 13
3.3 Writing the Scores Back to the Database 14
4. Interactive Dashboard **15**
4.1 Design Approach 15
4.2 Plotly Dashboard 16
4.3 R Shiny Application 20
4.4 Comparison of the Two Dashboards 25
5. Result Interpretation **26**
5.1 Customer Segments 26
5.2 Churn Risk and Revenue Exposure 26
5.3 Behavioural Drivers of Churn 27
5.4 Business Reading of the Results 27
5.5 Limits of the Interpretation 27
6. Conclusion, Limitations and Future Scope **28**
6.1 Conclusion 28
6.2 Limitations 28
6.3 Future Scope 29
8. References **30**

Abstract

This report presents the final phase of the customer segmentation project built on the Online Retail II dataset. DA1 cleaned the raw transactions, DA2 engineered customer-level features and trained twenty-four models across three prediction tasks, and this phase closes the work with a formal statistical comparison of the models, a consolidated customer scoring view, two interactive dashboards and the final interpretation of the results.

Five churn models were compared on the same 1,321-customer test set and the same five cross-validation folds. Four separate statistical procedures were applied: 95% confidence intervals for the mean fold score, paired t-tests and Wilcoxon signed-rank tests between every pair of models, pairwise McNemar tests on the test predictions, and bootstrap confidence intervals from 1,000 resamples of the test set. The Neural Network reached a test ROC-AUC of 0.8070 and Gradient Boosting 0.8054, and the paired t-test across folds shows no significant difference between them (mean difference 0.0003, $p = 0.8190$). Both are significantly better than Logistic Regression, Random Forest and XGBoost.

All 5,281 customers were then scored into one view combining churn probability, value tier and behavioural segment, from which a recommended action was derived. The scores were written back into the SQLite database as a fifth table. Total revenue exposed to churn, measured as lifetime spend weighted by churn probability, is GBP 3.67 million, of which GBP 1.06 million sits with the 569 customers marked for an urgent retention call.

Two dashboards were built: a self-contained Plotly HTML dashboard with five linked views, and an R Shiny application with reactive filters on risk band, value tier and segment. Both read the same exported data, so the figures shown match the report exactly. The closing chapters interpret the segment and risk results for a business reader and state the limitations of the work, in particular the 90-day churn definition and the use of historical spend as a proxy for future value.

Keywords: *customer segmentation, churn prediction, statistical significance testing, bootstrap confidence intervals, McNemar test, Plotly, R Shiny, SQLite, customer scoring.*

1. Introduction

1.1 Background

The project uses the Online Retail II dataset, which records transactions from a UK-based online gift retailer between December 2009 and December 2011. DA1 reduced 1,067,371 raw rows to 779,425 valid transactions. DA2 converted those into one record per customer using the RFM framework, defined three prediction targets, stored everything in a normalized SQLite database, and trained twenty-four models with tuning, threshold optimisation and SMOTE.

DA2 ended with two open questions. The leading models were separated by only thousandths of a point of ROC-AUC, and a single McNemar test on one split is thin evidence for choosing between them. The three model outputs, churn risk, high-value status and segment, were also still separate results rather than one view a business could act on. This phase answers both.

1.2 Objectives of This Phase

- Compare the final churn models using several statistical procedures rather than a single test.
- Quantify the uncertainty around each model's score using confidence intervals.
- Combine churn risk, customer value and segment into a single customer-level scoring view.
- Write the scoring view back into the database so it can be queried with SQL.
- Build an interactive dashboard in Plotly and a second one in R Shiny.
- Interpret the results and state the limitations and future scope of the work.

1.3 Tools and Environment

Component	Tool / Library	Purpose
Statistical analysis	Python 3 - SciPy, statsmodels	t-tests, Wilcoxon, McNemar
Modelling	scikit-learn, XGBoost	Model training and cross-validation
Resampling	NumPy	Bootstrap confidence intervals
Database	SQLite with sqlite3 (Python), DBI and RSQLite (R)	Storage and SQL analysis
Dashboard 1	Plotly (Python)	Self-contained interactive HTML
Dashboard 2	R Shiny with plotly, DT, bslib	Reactive web application
Environment	Google Colab, RStudio	Execution
Version control	Git and GitHub	Code and output management

Table 1: *Tools used in the final phase*

2. Final Performance Comparison and Statistical Analysis

DA2 ranked the models by ROC-AUC on one train-test split. That ranking is a starting point, not a conclusion: a difference of 0.0016 between two models can easily be produced by which customers happened to land in the test set. This chapter therefore measures the uncertainty around each score and tests whether the differences between models are larger than that uncertainty.

2.1 Experimental Setup

Five models were carried forward: the Neural Network (MLP), Gradient Boosting, Logistic Regression, Random Forest and XGBoost. All use the same twelve features, the same stratified 75/25 split of 3,960 training and 1,321 test customers, and the same scaler fitted on training data only. The same five stratified cross-validation folds are used for every model, which is what makes the paired tests valid.

Code:

```
[25]
✓ 0s
feature_cols = ["Recency", "Frequency", "Monetary", "AvgOrderValue",
                "AvgQuantity", "TotalQuantity", "OutlierOrders", "CountryGroup"]
df_model = pd.get_dummies(df[feature_cols + ["Churn"]], columns=["CountryGroup"], drop_first=True)

X = df_model.drop(columns=["Churn"])
y = df_model["Churn"].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)
X_all_s   = StandardScaler().fit_transform(X)

models = {
    "Neural Network (MLP)": MLPClassifier(hidden_layer_sizes=(32,16), max_iter=500, random_state=42),
    "Gradient Boosting":    GradientBoostingClassifier(random_state=42),
    "Logistic Regression":  LogisticRegression(max_iter=1000, random_state=42),
    "Random Forest":        RandomForestClassifier(n_estimators=200, random_state=42),
    "XGBoost":              XGBClassifier(eval_metric="logloss", random_state=42),
}

print(f"Train: {X_train.shape[0]} | Test: {X_test.shape[0]} | Features: {X.shape[1]}")
```

Output:

```
Train: 3960 | Test: 1321 | Features: 12
```

2.2 Consolidated Model Comparison

Threshold-dependent metrics measured on the test set were combined with the threshold-independent cross-validation scores into one comparison table.

Code:

```
[31]
✓ 0s
final_rows = []
for name, m in models.items():
    pred, proba = preds[name], probas[name]
    final_rows.append({
        "Model": name,
        "Accuracy": accuracy_score(yt, pred),
        "Precision": precision_score(yt, pred),
        "Recall": recall_score(yt, pred),
        "F1": f1_score(yt, pred),
        "Test ROC-AUC": roc_auc_score(yt, proba),
        "CV Mean AUC": fold_scores[name].mean(),
        "CV Std": fold_scores[name].std(ddof=1),
    })

final_df = pd.DataFrame(final_rows).sort_values("Test ROC-AUC", ascending=False).reset_index(drop=True)
final_df.to_csv("da3_final_comparison.csv", index=False)
print("Final consolidated comparison")
final_df.round(4)
```

Output:

Final consolidated comparison

	Model	Accuracy	Precision	Recall	F1	Test ROC-AUC	CV Mean AUC	CV Std
0	Neural Network (MLP)	0.7441	0.7595	0.8021	0.7802	0.8070	0.7964	0.0086
1	Gradient Boosting	0.7282	0.7369	0.8088	0.7712	0.8054	0.7960	0.0091
2	Logistic Regression	0.7366	0.7597	0.7821	0.7708	0.7973	0.7879	0.0120
3	Random Forest	0.7222	0.7378	0.7901	0.7631	0.7872	0.7823	0.0081
4	XGBoost	0.7055	0.7247	0.7741	0.7485	0.7722	0.7606	0.0100

The ordering matches DA2 exactly, which confirms that the pipeline reproduces. The Neural Network records the highest test ROC-AUC at 0.8070 and the highest accuracy at 0.7441, while Gradient Boosting has the highest recall at 0.8088, which matters for churn because a missed churner is the more expensive error. XGBoost is clearly the weakest of the five at 0.7722.

2.3 Cross-Validation Results

Five-fold stratified cross-validation was run on the full customer base so that every customer appears in a test fold exactly once. The individual fold scores matter here, not only their averages, because the paired tests in Section 2.5 operate directly on these five values per model.

Code:

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
fold_scores = {}

for name, m in models.items():
    scores = cross_val_score(m, X_all_s, y, cv=cv, scoring="roc_auc", n_jobs=-1)
    fold_scores[name] = scores
    print(f"{name:24s} {np.round(scores,4)}  mean={scores.mean():.4f}")

cv_folds_df = pd.DataFrame(fold_scores)
cv_folds_df.index = [f"Fold {i+1}" for i in range(5)]
cv_folds_df.to_csv("da3_cv_folds.csv")
cv_folds_df.round(4)
```

Output:

```
Neural Network (MLP)      [0.7967 0.8042 0.8    0.7993 0.7817]  mean=0.7964
Gradient Boosting         [0.7993 0.8006 0.798  0.8024 0.7799]  mean=0.7960
Logistic Regression       [0.792  0.7938 0.789  0.7975 0.7671]  mean=0.7879
Random Forest             [0.7849 0.785  0.7883 0.7855 0.768 ]  mean=0.7823
XGBoost                   [0.7509 0.7626 0.7701 0.77   0.7494]  mean=0.7606
```

	Neural Network (MLP)	Gradient Boosting	Logistic Regression	Random Forest	XGBoost
Fold 1	0.7967	0.7993	0.7920	0.7849	0.7509
Fold 2	0.8042	0.8006	0.7938	0.7850	0.7626
Fold 3	0.8000	0.7980	0.7890	0.7883	0.7701
Fold 4	0.7993	0.8024	0.7975	0.7855	0.7700
Fold 5	0.7817	0.7799	0.7671	0.7680	0.7494

Mean scores range from 0.7964 for the Neural Network down to 0.7606 for XGBoost, and every standard deviation is below 0.012. Fold 5 is the hardest fold for all five models, which indicates that the variation comes from the data partition rather than from any single model being unstable.

2.4 Confidence Intervals for Mean ROC-AUC

A 95% confidence interval was calculated for each model's mean fold score using the t-distribution with four degrees of freedom. The interval answers a different question from the mean itself: it describes the range in which the true performance is likely to lie.

Code:

```
[27]
✓ 0s
rows = []
for name, s in fold_scores.items():
    mean, sd, n = s.mean(), s.std(ddof=1), len(s)
    tcrit = stats.t.ppf(0.975, df=n-1)
    margin = tcrit * sd / np.sqrt(n)
    rows.append({"Model": name, "Mean_AUC": mean, "Std": sd,
                 "CI_Lower_95": mean - margin, "CI_Upper_95": mean + margin})

ci_df = pd.DataFrame(rows).sort_values("Mean_AUC", ascending=False).reset_index(drop=True)
ci_df.to_csv("da3_confidence_intervals.csv", index=False)
print("95% confidence intervals for mean ROC-AUC (5 folds)")
ci_df.round(4)
```

Output:

95% confidence intervals for mean ROC-AUC (5 folds)

	Model	Mean_AUC	Std	CI_Lower_95	CI_Upper_95
0	Neural Network (MLP)	0.7964	0.0086	0.7856	0.8071
1	Gradient Boosting	0.7960	0.0091	0.7847	0.8074
2	Logistic Regression	0.7879	0.0120	0.7729	0.8028
3	Random Forest	0.7823	0.0081	0.7722	0.7924
4	XGBoost	0.7606	0.0100	0.7481	0.7731

The intervals for the Neural Network (0.7856 to 0.8071) and Gradient Boosting (0.7847 to 0.8074) overlap almost completely, so the two cannot be separated on this evidence. The

XGBoost interval, 0.7481 to 0.7731, does not overlap with any other model, which makes it the weakest of the five on this dataset.

2.5 Paired Significance Tests Across Folds

Because every model was evaluated on identical folds, the scores are paired and the differences can be tested directly. Two tests were applied to each of the ten pairs: the paired t-test, which assumes the differences are approximately normal, and the Wilcoxon signed-rank test, which does not. With only five folds the Wilcoxon test cannot produce a p-value below 0.0625, so it is reported as supporting evidence rather than as the decision rule.

Code:

```
[28]
✓ 0s
names = list(fold_scores.keys())
results = []

for i in range(len(names)):
    for j in range(i+1, len(names)):
        a, b = fold_scores[names[i]], fold_scores[names[j]]
        t_stat, t_p = stats.ttest_rel(a, b)
        try:
            w_stat, w_p = stats.wilcoxon(a, b)
        except ValueError:
            w_stat, w_p = np.nan, np.nan
        results.append({
            "Model A": names[i], "Model B": names[j],
            "Mean Diff": a.mean() - b.mean(),
            "t-statistic": t_stat, "t p-value": t_p,
            "Wilcoxon p": w_p,
            "Significant (5%)": "Yes" if t_p < 0.05 else "No"
        })

pairwise_df = pd.DataFrame(results)
pairwise_df.to_csv("da3_pairwise_tests.csv", index=False)
print("Paired t-test and Wilcoxon signed-rank test on 5-fold ROC-AUC")
pairwise_df.round(4)
```

Output:

Paired t-test and Wilcoxon signed-rank test on 5-fold ROC-AUC

	Model A	Model B	Mean Diff	t-statistic	t p-value	Wilcoxon p	Significant (5%)
0	Neural Network (MLP)	Gradient Boosting	0.0003	0.2444	0.8190	1.0000	No
1	Neural Network (MLP)	Logistic Regression	0.0085	3.6655	0.0215	0.0625	Yes
2	Neural Network (MLP)	Random Forest	0.0141	10.2558	0.0005	0.0625	Yes
3	Neural Network (MLP)	XGBoost	0.0358	10.6909	0.0004	0.0625	Yes
4	Gradient Boosting	Logistic Regression	0.0082	6.0580	0.0037	0.0625	Yes
5	Gradient Boosting	Random Forest	0.0137	10.7181	0.0004	0.0625	Yes
6	Gradient Boosting	XGBoost	0.0355	9.7741	0.0006	0.0625	Yes
7	Logistic Regression	Random Forest	0.0056	2.2752	0.0852	0.1875	No
8	Logistic Regression	XGBoost	0.0273	6.3706	0.0031	0.0625	Yes
9	Random Forest	XGBoost	0.0217	6.6798	0.0026	0.0625	Yes

Eight of the ten comparisons are significant at the 5% level. The two that are not are the Neural Network against Gradient Boosting, where the mean difference is 0.0003 with p = 0.8190, and Logistic Regression against Random Forest, where p = 0.0852. In both cases the models are close enough that fold-to-fold variation swamps the difference between them.

2.6 Pairwise McNemar Tests on the Test Set

McNemar's test works differently. Instead of comparing average scores, it looks at the individual test customers where two models disagree and asks whether the disagreement is lopsided. Only the two off-diagonal counts matter: the customers one model gets right and the other gets wrong.

Code:

```
[29]
✓ 13s
preds = {}
for name, m in models.items():
    m.fit(X_train_s, y_train)
    preds[name] = m.predict(X_test_s)

yt = y_test.values
mc_rows = []
for i in range(len(names)):
    for j in range(i+1, len(names)):
        a = (preds[names[i]] == yt).astype(int)
        b = (preds[names[j]] == yt).astype(int)
        n01 = int(((a==1)&(b==0)).sum())      # only A correct
        n10 = int(((a==0)&(b==1)).sum())      # only B correct
        tbl = [[int(((a==1)&(b==1)).sum()), n01],
               [n10, int(((a==0)&(b==0)).sum())]]
        res = mcnemar(tbl, exact=False, correction=True)
        mc_rows.append({"Model A": names[i], "Model B": names[j],
                        "A only correct": n01, "B only correct": n10,
                        "Statistic": res.statistic, "p-value": res.pvalue,
                        "Significant (5%)": "Yes" if res.pvalue < 0.05 else "No"})

mcnemar_df = pd.DataFrame(mc_rows)
mcnemar_df.to_csv("da3_mcnemar_matrix.csv", index=False)
print("Pairwise McNemar tests on the 1,321 test customers")
mcnemar_df.round(4)
```

Output:

Pairwise McNemar tests on the 1,321 test customers

	Model A	Model B	A only correct	B only correct	Statistic	p-value	Significant (5%)
0	Neural Network (MLP)	Gradient Boosting	58	37	4.2105	0.0402	Yes
1	Neural Network (MLP)	Logistic Regression	57	47	0.7788	0.3775	No
2	Neural Network (MLP)	Random Forest	92	63	5.0581	0.0245	Yes
3	Neural Network (MLP)	XGBoost	137	86	11.2108	0.0008	Yes
4	Gradient Boosting	Logistic Regression	54	65	0.8403	0.3593	No
5	Gradient Boosting	Random Forest	76	68	0.3403	0.5597	No
6	Gradient Boosting	XGBoost	112	82	4.3351	0.0373	Yes
7	Logistic Regression	Random Forest	103	84	1.7326	0.1881	No
8	Logistic Regression	XGBoost	142	101	6.5844	0.0103	Yes
9	Random Forest	XGBoost	97	75	2.5640	0.1093	No

Here the Neural Network does separate from Gradient Boosting: it is correct on 58 customers that Gradient Boosting misclassifies, against 37 in the other direction, giving p = 0.0402. The same test finds no significant difference between Gradient Boosting and either Logistic Regression or Random Forest, and the largest gap of all is the Neural Network against XGBoost at p = 0.0008.

2.7 Bootstrap Confidence Intervals

The test set was resampled with replacement 1,000 times and ROC-AUC recomputed on each resample. The 2.5th and 97.5th percentiles of those 1,000 values give an empirical confidence interval that makes no distributional assumption.

Code:

```
[30]
✓ 10s
probas = {name: m.predict_proba(X_test_s)[:,1] for name, m in models.items()}
rng = np.random.default_rng(42)
n = len(yt)

boot_rows = []
for name, p in probas.items():
    aucs = []
    for _ in range(1000):
        idx = rng.integers(0, n, n)
        if len(np.unique(yt[idx])) < 2:
            continue
        aucs.append(roc_auc_score(yt[idx], p[idx]))
    lo, hi = np.percentile(aucs, [2.5, 97.5])
    boot_rows.append({"Model": name, "Test_AUC": roc_auc_score(yt, p),
                      "Boot_CI_Lower": lo, "Boot_CI_Upper": hi})

boot_df = pd.DataFrame(boot_rows).sort_values("Test_AUC", ascending=False).reset_index(drop=True)
boot_df.to_csv("da3_bootstrap_ci.csv", index=False)
print("Bootstrap 95% CI for test ROC-AUC (1,000 resamples)")
boot_df.round(4)
```

Output:

```
Bootstrap 95% CI for test ROC-AUC (1,000 resamples)
```

	Model	Test_AUC	Boot_CI_Lower	Boot_CI_Upper
0	Neural Network (MLP)	0.8070	0.7835	0.8273
1	Gradient Boosting	0.8054	0.7843	0.8283
2	Logistic Regression	0.7973	0.7721	0.8207
3	Random Forest	0.7872	0.7644	0.8105
4	XGBoost	0.7722	0.7449	0.7973

Every interval is roughly 0.045 wide, which is far larger than the 0.0016 that separates the top two models. Read on its own, the bootstrap says the single-split ranking should not be treated as a firm ordering at the top of the table.

2.8 Reconciling the Two Testing Approaches

The paired t-test across folds and McNemar's test on the test set disagree about the Neural Network and Gradient Boosting. The disagreement is informative rather than contradictory, because the two tests measure different things.

- The paired t-test compares average ranking quality (ROC-AUC) across five different partitions of the data. It asks whether one model would rank customers better on a newly drawn sample.
- McNemar's test compares hard 0/1 predictions on one fixed set of customers at a fixed threshold. It asks whether the two models make the same mistakes on these specific people.

So the Neural Network makes measurably different errors from Gradient Boosting on this particular test set, but across resampled folds its overall ranking ability is not better. Both readings are supported by the bootstrap intervals, which overlap heavily.

Conclusion of the statistical analysis: The Neural Network and Gradient Boosting models are statistically indistinguishable in ranking performance and should therefore be considered equivalent. Both models perform significantly better than Logistic Regression, Random Forest, and XGBoost, based on the paired fold tests.

2.9 Final Model Selection

Since the two leading models cannot be separated statistically, selection falls to practical criteria. Gradient Boosting was chosen as the production model for three reasons: it has the highest recall of the five models at 0.8088, which is the metric that matters most for retention; it was the model tuned in DA2, where grid search and threshold optimisation lifted its recall to 0.873; and it trains faster and is easier to retrain than the neural network, with fewer settings that affect stability.

Criterion	Neural Network (MLP)	Gradient Boosting	Chosen
Test ROC-AUC	0.8070	0.8054	Equivalent
Mean CV ROC-AUC	0.7964	0.7960	Equivalent (p = 0.8190)
Recall	0.8021	0.8088	Gradient Boosting
Tuned in DA2	No	Yes, threshold 0.453	Gradient Boosting
Training cost and stability	Higher, sensitive to settings	Lower, stable	Gradient Boosting

Table 2: *Basis for selecting the production model*

3. Consolidated Customer Scoring View

DA2 produced three separate outputs: a churn probability, a high-value flag and a segment number. On their own none of them tells anyone what to do about a particular customer. This chapter combines them into one record per customer with a recommended action attached.

3.1 Combining Risk, Value and Segment

Every one of the 5,281 customers was scored with the tuned Gradient Boosting model and the optimised threshold of 0.453 from DA2. Three derived fields were then added.

Field	Definition
Risk band	Low below 0.3, Medium 0.3 to 0.6, High above 0.6 churn probability
Value tier	Quartiles of lifetime spend: Bronze, Silver, Gold, Platinum
Revenue at risk	Lifetime spend multiplied by churn probability

Table 3: *Fields derived for the scoring view*

Revenue at risk is the field that makes the view useful. A customer with a 90% churn probability and GBP 200 of lifetime spend is a smaller problem than a customer with a 60% probability and GBP 50,000 of spend, and ranking by probability alone would put them the wrong way round.

Code:

```
[32]
✓ 0s
# Gradient Boosting with the tuned threshold from DA2
gb = models["Gradient Boosting"]
all_proba = gb.predict_proba(scaler.transform(X))[:, 1]

scores = pd.DataFrame({
    "customer_id": df["Customer ID"].astype(int),
    "segment": df["Segment"].astype(int),
    "monetary": df["Monetary"].round(2),
    "frequency": df["Frequency"].astype(int),
    "recency": df["Recency"].round(1),
    "high_value": df["HighValue"].astype(int),
    "churn_actual": df["Churn"].astype(int),
    "churn_probability": all_proba.round(4),
})
scores["churn_predicted"] = (scores["churn_probability"] >= 0.453).astype(int)
scores["risk_band"] = pd.cut(scores["churn_probability"], bins=[0, 0.3, 0.6, 1.0],
                             labels=["Low", "Medium", "High"]).astype(str)
scores["value_tier"] = pd.qcut(scores["monetary"], q=4,
                               labels=["Bronze", "Silver", "Gold", "Platinum"]).astype(str)

# revenue exposed per customer = how much they are worth x how likely they are to leave
scores["revenue_at_risk"] = (scores["monetary"] * scores["churn_probability"]).round(2)

def action(r):
    if r.risk_band == "High":
        return "Urgent retention call" if r.value_tier in ("Gold","Platinum") else "Automated win-back offer"
    if r.risk_band == "Medium":
        return "Proactive engagement" if r.value_tier in ("Gold","Platinum") else "Monitor"
    return "Loyalty / upsell" if r.value_tier in ("Gold","Platinum") else "Maintain"

scores["recommended_action"] = scores.apply(action, axis=1)
scores = scores.sort_values("revenue_at_risk", ascending=False).reset_index(drop=True)
scores.to_csv("da3_customer_scores.csv", index=False)

print("Customers scored:", len(scores))
print(scores["recommended_action"].value_counts(), "\n")
print("Total revenue at risk: GBP", f"{scores['revenue_at_risk'].sum():,.0f}")
scores.head(10)
```

Output:

```
Customers scored: 5281
recommended_action
Automated win-back offer    2046
Proactive engagement        1091
Loyalty / upsell             988
Monitor                      586
Urgent retention call        569
Maintain                       9
Name: count, dtype: int64

Total revenue at risk: GBP 3,668,134
```

	customer_id	segment	monetary	frequency	recency	high_value	churn_actual	churn_probability	churn_predicted	risk_band	value_tier	revenue_at_risk	recommended_action
0	12346	2	77556.46	12	235.1	1	1	0.9023	1	High	Platinum	69979.19	Urgent retention call
1	16754	2	65600.07	29	281.8	1	1	0.8149	1	High	Platinum	53376.01	Urgent retention call
2	15749	2	44634.30	3	145.0	1	1	0.8328	1	High	Platinum	37068.17	Urgent retention call
3	18102	4	456780.49	112	8.0	1	0	0.0770	0	Low	Platinum	35172.10	Loyalty / upsell
4	14646	4	426699.12	120	2.1	1	0	0.0812	0	Low	Platinum	34647.97	Loyalty / upsell
5	17850	4	51208.87	155	281.9	1	1	0.6654	1	High	Platinum	34074.38	Urgent retention call
6	15098	3	39916.50	3	91.9	1	1	0.7863	1	High	Platinum	31388.34	Urgent retention call
7	13902	1	34095.26	5	542.0	1	1	0.9091	1	High	Platinum	30998.00	Urgent retention call
8	13093	2	54144.25	55	185.0	1	1	0.5667	1	Medium	Platinum	30683.55	Proactive engagement
9	14156	4	294916.33	143	2.9	1	0	0.0759	0	Low	Platinum	22384.15	Loyalty / upsell

Scoring all 5,281 customers gives a total exposure of GBP 3,668,134. The highest single entry is customer 12346, with GBP 77,556 of lifetime spend and a churn probability of 0.902, which places GBP 69,979 at risk from that one account.

3.2 Recommended Action Matrix

Risk band and value tier together decide the recommended action. The logic is deliberately simple so that it can be explained to a non-technical reader and changed without retraining anything.

Risk band	Gold or Platinum customer	Bronze or Silver customer
High	Urgent retention call	Automated win-back offer
Medium	Proactive engagement	Monitor
Low	Loyalty / upsell	Maintain

Table 4: *Action assigned to each combination of risk and value*

Two observations follow from the resulting counts. The 2,046 customers marked for an automated win-back have the highest average risk at 0.802 but hold only GBP 636,537 of lifetime value between them, so an inexpensive automated channel is appropriate. The 569 customers marked for an urgent call carry GBP 1,061,677 of revenue at risk, almost a third of the total, which justifies individual human contact.

3.3 Writing the Scores Back to the Database

The scoring view was written into the SQLite database as a fifth table, customer_scores, with a foreign key to the customers table and indexes on customer_id and recommended_action. A grouped query over that table returns the summary of customers and revenue by recommended action.

Code:

```
conn = sqlite3.connect("da2_retail_normalized.sqlite")
cur = conn.cursor()
cur.execute("DROP TABLE IF EXISTS customer_scores")
cur.execute("""
CREATE TABLE customer_scores (
  score_id          INTEGER PRIMARY KEY AUTOINCREMENT,
  customer_id       INTEGER NOT NULL,
  segment           INTEGER,
  monetary          REAL,
  frequency         INTEGER,
  recency           REAL,
  high_value        INTEGER CHECK (high_value IN (0,1)),
  churn_actual      INTEGER CHECK (churn_actual IN (0,1)),
  churn_probability REAL,
  churn_predicted   INTEGER CHECK (churn_predicted IN (0,1)),
  risk_band         TEXT,
  value_tier        TEXT,
  revenue_at_risk   REAL,
  recommended_action TEXT,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id))""")
conn.commit()

scores.to_sql("customer_scores", conn, if_exists="append", index=False)
cur.execute("CREATE INDEX idx_scores_customer ON customer_scores(customer_id)")
cur.execute("CREATE INDEX idx_scores_action   ON customer_scores(recommended_action)")
conn.commit()

print(pd.read_sql("""
SELECT recommended_action, COUNT(*) AS customers,
       ROUND(AVG(churn_probability),3) AS avg_risk,
       ROUND(SUM(monetary),2)          AS lifetime_value,
       ROUND(SUM(revenue_at_risk),2)   AS revenue_at_risk
FROM customer_scores GROUP BY recommended_action
ORDER BY revenue_at_risk DESC""", conn).to_string(index=False))
```

Output:

```
recommended_action  customers  avg_risk  lifetime_value  revenue_at_risk
Urgent retention call        569     0.733      1383815.30       1061677.31
Proactive engagement        1091     0.449      2256525.77        996157.84
Loyalty / upsell             980     0.153      9368647.89        975470.17
Automated win-back offer    2046     0.882       636537.07        494450.50
Monitor                      586     0.494       282776.80        139625.41
Maintain                       9     0.252         3853.17           752.84
```

The database now holds five tables and nine indexes, and the full chain R to SQLite to Python to SQLite remains intact.

Table	Rows	Written by	Added in
customers	5,281	R	DA2
products	4,631	R	DA2
transactions	752,407	R	DA2
ml_predictions	5,281	Python	DA2
customer_scores	5,281	Python	DA3

Table 5: *Final state of the database*

4. Interactive Dashboard

4.1 Design Approach

Two dashboards were built rather than one. The Plotly dashboard is a single HTML file that opens in any browser with no software installed, which makes it easy to submit and to demonstrate. The Shiny application adds genuine server-side reactivity: filters recompute the underlying data and every figure updates together. Between them they cover both readings of the requirement.

Both dashboards read the same CSV files exported from the analysis notebook, so no figure on either dashboard can drift from the numbers in this report. The colour palette was checked for colour-blind separation, every chart carries hover tooltips, and each chart is paired with a table so that values can also be read directly.

Code:

```
[38]
✓ 0s
import sqlite3, pandas as pd

df     = pd.read_csv("customer_features.csv")
scores = pd.read_csv("da3_customer_scores.csv")

# 1. segment profiles
seg_feats = ["Recency","Frequency","Monetary","AvgOrderValue","AvgQuantity","TotalQuantity"]
seg_profile = df.groupby("Segment")[seg_feats].mean().round(2)
seg_profile["Customers"] = df.groupby("Segment").size()
seg_profile["ChurnRate"] = (df.groupby("Segment")["Churn"].mean()*100).round(1)
seg_profile.reset_index().to_csv("da3_segment_profiles.csv", index=False)

# 2. risk bands
(scores.groupby("risk_band")
       .agg(customers=("customer_id","size"),
            avg_churn_prob=("churn_probability","mean"),
            avg_lifetime_value=("monetary","mean"),
            total_value=("monetary","sum"),
            revenue_at_risk=("revenue_at_risk","sum"))
       .round(2).reset_index()
       .to_csv("da3_risk_bands.csv", index=False))

# 3. monthly revenue from the database
conn = sqlite3.connect("da2_retail_normalized.sqlite")
pd.read_sql("""
SELECT SUBSTR(invoice_date,1,7) AS month,
       COUNT(DISTINCT invoice)  AS orders,
       ROUND(SUM(revenue),2)    AS revenue
FROM transactions GROUP BY month ORDER BY month""", conn).to_csv("da3_monthly_revenue.csv", index=False)
conn.close()

import os
need = ["da3_final_comparison.csv","da3_cv_folds.csv","da3_bootstrap_ci.csv",
        "da3_pairwise_tests.csv","da3_mcnemar_matrix.csv","da3_roc_points.csv",
        "da3_customer_scores.csv","da3_segment_profiles.csv","da3_risk_bands.csv",
        "da3_monthly_revenue.csv","customer_features.csv"]
for f in need:
    print(("OK   " if os.path.exists(f) else "MISS "), f)
```

Output:

```
OK    da3_final_comparison.csv
OK    da3_cv_folds.csv
OK    da3_bootstrap_ci.csv
OK    da3_pairwise_tests.csv
OK    da3_mcnemar_matrix.csv
OK    da3_roc_points.csv
OK    da3_customer_scores.csv
OK    da3_segment_profiles.csv
OK    da3_risk_bands.csv
OK    da3_monthly_revenue.csv
OK    customer_features.csv
```

4.2 Plotly Dashboard

The file DA3_Dashboard.html contains five summary tiles across the top and five tabs below them: Overview, Model comparison, Segments, Revenue and Retention list. The Plotly library is embedded inside the file, so the dashboard works offline and needs no server.

The Overview tab carries revenue at risk by recommended action together with customers and revenue by risk band. Model comparison holds the ROC-AUC chart with bootstrap intervals, the cross-validation box plot, the ROC curves and the three statistical test tables. Segments plots recency against spend with point size set by order frequency. Revenue shows the monthly series and the exposure by value tier, and Retention list names the twenty highest-value customers in the high-risk band.

Code:

```
[36]
✓ 0s
import pandas as pd, numpy as np, plotly.graph_objects as go, plotly.express as px
from plotly.offline import get_plotlyjs

# ---------- load exports ----------
final_df   = pd.read_csv("da3_final_comparison.csv")
folds      = pd.read_csv("da3_cv_folds.csv", index_col=0)
boot       = pd.read_csv("da3_bootstrap_ci.csv")
pairwise   = pd.read_csv("da3_pairwise_tests.csv")
mcn        = pd.read_csv("da3_mcnemar_matrix.csv")
roc_pts    = pd.read_csv("da3_roc_points.csv")
scores     = pd.read_csv("da3_customer_scores.csv")
segprof    = pd.read_csv("da3_segment_profiles.csv")
monthly    = pd.read_csv("da3_monthly_revenue.csv")
feats      = pd.read_csv("customer_features.csv")

# ---------- palette (validated for colour-blind separation) ----------
CAT   = ["#2A6FB0", "#C65A1E", "#0E8F86", "#8E4EA8", "#B8902A"]
RISK  = {"Low": "#0E8F86", "Medium": "#B8902A", "High": "#C65A1E"}
SURF, INK, MUTED, GRID = "#fcfcfb", "#33312e", "#6f6a62", "#eceae4"

def style(fig, h=430, title=""):
    fig.update_layout(template="simple_white", height=h,
        title=dict(text=title, font=dict(size=16, color=INK), x=0, xanchor="left"),
        margin=dict(l=70, r=30, t=55, b=55),
        font=dict(family="Segoe UI, Inter, Arial", size=13, color=INK),
        paper_bgcolor=SURF, plot_bgcolor=SURF,
        legend=dict(orientation="h", y=-0.18, x=0, font=dict(color=MUTED)))
    fig.update_xaxes(showgrid=False, linecolor="#d9d4cb", ticks="outside", tickcolor="#d9d4cb")
    fig.update_yaxes(gridcolor=GRID, linecolor="#d9d4cb", zeroline=False)
    return fig

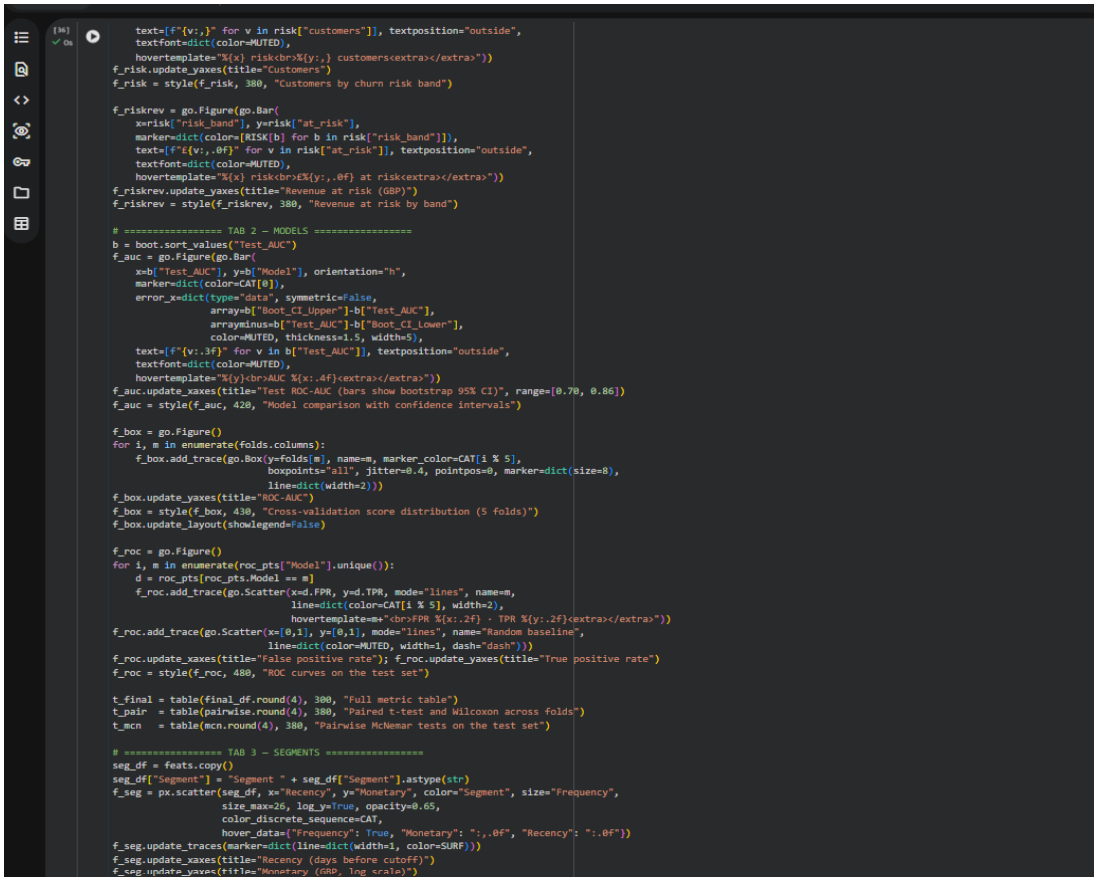
def table(dfx, h=330, title="", widths=None):
    fig = go.Figure(go.Table(
        columnwidth=widths,
        header=dict(values=[f"<b>{c}</b>" for c in dfx.columns], fill_color="#efece5",
                    align="left", font=dict(color=INK, size=12), height=30),
        cells=dict(values=[dfx[c] for c in dfx.columns], fill_color=SURF, align="left",
                   font=dict(color=INK, size=12), height=26,
                   line=dict(color=GRID, width=1))))
    return style(fig, h, title)

# ================ TAB 1 — OVERVIEW ================
act = (scores.groupby("recommended_action")
      .agg(customers=("customer_id","size"), revenue_at_risk=("revenue_at_risk","sum"))
      .reset_index().sort_values("revenue_at_risk"))

f_act = go.Figure(go.Bar(
    x=act["revenue_at_risk"], y=act["recommended_action"], orientation="h",
    marker=dict(color=CAT[0], line=dict(width=0)),
    text=[f"£{v:,.0f}  ({c:,} customers)" for v, c in zip(act.revenue_at_risk, act.customers)],
    textposition="outside", textfont=dict(color=MUTED, size=12),
    hovertemplate="%{y}<br>Revenue at risk £%{x:,.0f}<extra></extra>"))
f_act.update_xaxes(title="Revenue at risk (GBP)", range=[0, act.revenue_at_risk.max()*1.45])
f_act = style(f_act, 430, "Revenue exposed by recommended action")

risk = (scores.groupby("risk_band")
       .agg(customers=("customer_id","size"), lifetime=("monetary","sum"),
            at_risk=("revenue_at_risk","sum"), avg_prob=("churn_probability","mean"))
       .reindex(["Low","Medium","High"]).reset_index())

f_risk = go.Figure(go.Bar(
    x=risk["risk_band"], y=risk["customers"],
    marker=dict(color=[RISK[b] for b in risk["risk_band"]]),
```



```
        text=[f"{v:,}" for v in risk["customers"]], textposition="outside",
        textfont=dict(color=MUTED),
        hovertemplate="%{x} risk<br>%{y:,} customers<extra></extra>"))
f_risk.update_yaxes(title="Customers")
f_risk = style(f_risk, 380, "Customers by churn risk band")

f_riskrev = go.Figure(go.Bar(
    x=risk["risk_band"], y=risk["at_risk"],
    marker=dict(color=[RISK[b] for b in risk["risk_band"]]),
    text=[f"£{v:,.0f}" for v in risk["at_risk"]], textposition="outside",
    textfont=dict(color=MUTED),
    hovertemplate="%{x} risk<br>£%{y:,.0f} at risk<extra></extra>"))
f_riskrev.update_yaxes(title="Revenue at risk (GBP)")
f_riskrev = style(f_riskrev, 380, "Revenue at risk by band")

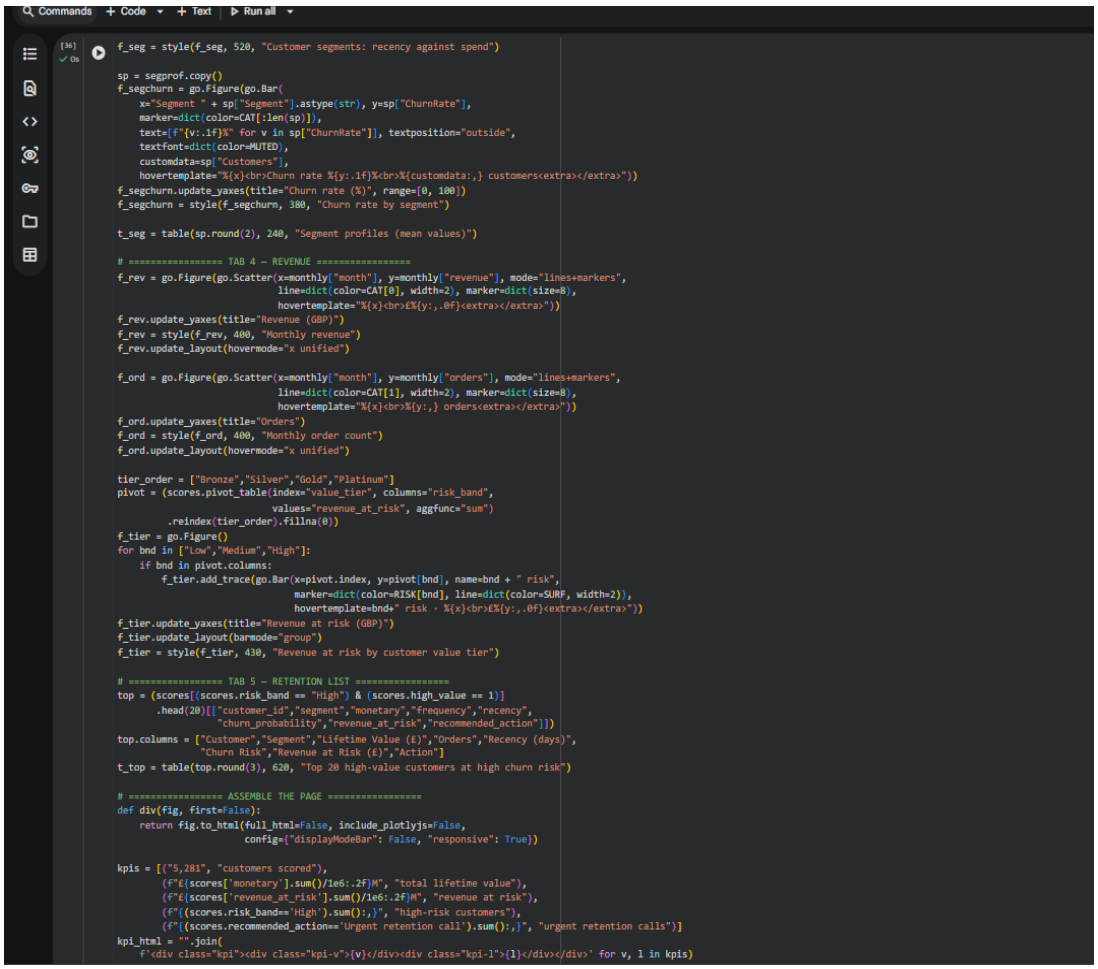
# ================ TAB 2 — MODELS ================
b = boot.sort_values("Test_AUC")
f_auc = go.Figure(go.Bar(
    x=b["Test_AUC"], y=b["Model"], orientation="h",
    marker=dict(color=CAT[0]),
    error_x=dict(type="data", symmetric=False,
                 array=b["Boot_CI_Upper"]-b["Test_AUC"],
                 arrayminus=b["Test_AUC"]-b["Boot_CI_Lower"],
                 color=MUTED, thickness=1.5, width=5),
    text=[f"{v:.3f}" for v in b["Test_AUC"]], textposition="outside",
    textfont=dict(color=MUTED),
    hovertemplate="%{y}<br>AUC %{x:.4f}<extra></extra>"))
f_auc.update_xaxes(title="Test ROC-AUC (bars show bootstrap 95% CI)", range=[0.70, 0.86])
f_auc = style(f_auc, 420, "Model comparison with confidence intervals")

f_box = go.Figure()
for i, m in enumerate(folds.columns):
    f_box.add_trace(go.Box(y=folds[m], name=m, marker_color=CAT[i % 5],
                           boxpoints="all", jitter=0.4, pointpos=0, marker=dict(size=8),
                           line=dict(width=2)))
f_box.update_yaxes(title="ROC-AUC")
f_box = style(f_box, 430, "Cross-validation score distribution (5 folds)")
f_box.update_layout(showlegend=False)

f_roc = go.Figure()
for i, m in enumerate(roc_pts["Model"].unique()):
    d = roc_pts[roc_pts.Model == m]
    f_roc.add_trace(go.Scatter(x=d.FPR, y=d.TPR, mode="lines", name=m,
                               line=dict(color=CAT[i % 5], width=2),
                               hovertemplate=m+"<br>FPR %{x:.2f} · TPR %{y:.2f}<extra></extra>"))
f_roc.add_trace(go.Scatter(x=[0,1], y=[0,1], mode="lines", name="Random baseline",
                           line=dict(color=MUTED, width=1, dash="dash")))
f_roc.update_xaxes(title="False positive rate"); f_roc.update_yaxes(title="True positive rate")
f_roc = style(f_roc, 480, "ROC curves on the test set")

t_final = table(final_df.round(4), 300, "Full metric table")
t_pair  = table(pairwise.round(4), 380, "Paired t-test and Wilcoxon across folds")
t_mcn   = table(mcn.round(4), 380, "Pairwise McNemar tests on the test set")

# ================ TAB 3 — SEGMENTS ================
seg_df = feats.copy()
seg_df["Segment"] = "Segment " + seg_df["Segment"].astype(str)
f_seg = px.scatter(seg_df, x="Recency", y="Monetary", color="Segment", size="Frequency",
                   size_max=26, log_y=True, opacity=0.65,
                   color_discrete_sequence=CAT,
                   hover_data={"Frequency": True, "Monetary": ":,.0f", "Recency": ":.0f"})
f_seg.update_traces(marker=dict(line=dict(width=1, color=SURF)))
f_seg.update_xaxes(title="Recency (days before cutoff)")
f_seg.update_yaxes(title="Monetary (GBP, log scale)")
```



```
Commands  + Code  + Text  ▷ Run all

f_seg = style(f_seg, 520, "Customer segments: recency against spend")

sp = segprof.copy()
f_segchurn = go.Figure(go.Bar(
    x="Segment " + sp["Segment"].astype(str), y=sp["ChurnRate"],
    marker=dict(color=CAT[:len(sp)]),
    text=[f"{v:.1f}%" for v in sp["ChurnRate"]], textposition="outside",
    textfont=dict(color=MUTED),
    customdata=sp["Customers"],
    hovertemplate="%{x}<br>Churn rate %{y:.1f}%<br>%{customdata:,} customers<extra></extra>"))
f_segchurn.update_yaxes(title="Churn rate (%)", range=[0, 100])
f_segchurn = style(f_segchurn, 380, "Churn rate by segment")

t_seg = table(sp.round(2), 240, "Segment profiles (mean values)")

# ================ TAB 4 — REVENUE ================
f_rev = go.Figure(go.Scatter(x=monthly["month"], y=monthly["revenue"], mode="lines+markers",
                             line=dict(color=CAT[0], width=2), marker=dict(size=8),
                             hovertemplate="%{x}<br>£%{y:,.0f}<extra></extra>"))
f_rev.update_yaxes(title="Revenue (GBP)")
f_rev = style(f_rev, 400, "Monthly revenue")
f_rev.update_layout(hovermode="x unified")

f_ord = go.Figure(go.Scatter(x=monthly["month"], y=monthly["orders"], mode="lines+markers",
                             line=dict(color=CAT[1], width=2), marker=dict(size=8),
                             hovertemplate="%{x}<br>%{y:,} orders<extra></extra>"))
f_ord.update_yaxes(title="Orders")
f_ord = style(f_ord, 400, "Monthly order count")
f_ord.update_layout(hovermode="x unified")

tier_order = ["Bronze","Silver","Gold","Platinum"]
pivot = (scores.pivot_table(index="value_tier", columns="risk_band",
                            values="revenue_at_risk", aggfunc="sum")
         .reindex(tier_order).fillna(0))
f_tier = go.Figure()
for bnd in ["Low","Medium","High"]:
    if bnd in pivot.columns:
        f_tier.add_trace(go.Bar(x=pivot.index, y=pivot[bnd], name=bnd + " risk",
                                marker=dict(color=RISK[bnd], line=dict(color=SURF, width=2)),
                                hovertemplate=bnd+" risk · %{x}<br>£%{y:,.0f}<extra></extra>"))
f_tier.update_yaxes(title="Revenue at risk (GBP)")
f_tier.update_layout(barmode="group")
f_tier = style(f_tier, 430, "Revenue at risk by customer value tier")

# ================ TAB 5 — RETENTION LIST ================
top = (scores[(scores.risk_band == "High") & (scores.high_value == 1)]
       .head(20)[["customer_id","segment","monetary","frequency","recency",
                  "churn_probability","revenue_at_risk","recommended_action"]])
top.columns = ["Customer","Segment","Lifetime Value (£)","Orders","Recency (days)",
               "Churn Risk","Revenue at Risk (£)","Action"]
t_top = table(top.round(3), 620, "Top 20 high-value customers at high churn risk")

# ================ ASSEMBLE THE PAGE ================
def div(fig, first=False):
    return fig.to_html(full_html=False, include_plotlyjs=False,
                       config={"displayModeBar": False, "responsive": True})

kpis = [("5,281", "customers scored"),
        (f"£{scores['monetary'].sum()/1e6:.2f}M", "total lifetime value"),
        (f"£{scores['revenue_at_risk'].sum()/1e6:.2f}M", "revenue at risk"),
        (f"{(scores.risk_band=='High').sum():,}", "high-risk customers"),
        (f"{(scores.recommended_action=='Urgent retention call').sum():,}", "urgent retention calls")]
kpi_html = "".join(
    f'<div class="kpi"><div class="kpi-v">{v}</div><div class="kpi-l">{l}</div></div>' for v, l in kpis)
```

```
TABS = [("overview", "Overview"), ("models", "Model comparison"),
        ("segments", "Segments"), ("revenue", "Revenue"), ("retention", "Retention list")]
tabbar = "".join(f'<button class="tab{" on" if i==0 else ""}" data-t="{k}">{n}</button>'
                 for i, (k, n) in enumerate(TABS))

html = f"""<!DOCTYPE html><html lang="en"><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Customer Analytics Dashboard — DA3</title>
<script>{get_plotlyjs()}</script>
<style>
 :root {{ --surf:{SURF}; --ink:{INK}; --muted:{MUTED}; --grid:{GRID}; --card:#ffffff; }}
 *{{box-sizing:border-box}}
 body{{margin:0;background:#f4f1ea;color:var(--ink);
   font-family:"Segoe UI",Inter,Arial,sans-serif}}
 header{{background:var(--card);padding:26px 32px 18px;border-bottom:1px solid #e3ded4}}
 h1{{margin:0 0 4px;font-size:25px;font-weight:600}}
 .sub{{color:var(--muted);font-size:14px}}
 .kpis{{display:flex;flex-wrap:wrap;gap:14px;padding:20px 32px 6px}}
 .kpi{{background:var(--card);border:1px solid #e6e1d7;border-radius:10px;
   padding:16px 22px;min-width:168px;flex:1}}
 .kpi-v{{font-size:27px;font-weight:600;color:#2A6FB0}}
 .kpi-l{{font-size:12.5px;color:var(--muted);margin-top:3px}}
 .tabs{{display:flex;gap:8px;flex-wrap:wrap;padding:16px 32px 0}}
 .tab{{background:transparent;border:1px solid #ddd6ca;border-radius:999px;
   padding:8px 18px;font-size:14px;color:var(--muted);cursor:pointer}}
 .tab.on{{background:#2A6FB0;border-color:#2A6FB0;color:#fff}}
 .panel{{display:none;padding:18px 32px 44px}} .panel.on{{display:block}}
 .card{{background:var(--card);border:1px solid #e6e1d7;border-radius:10px;
   padding:10px 14px;margin-bottom:18px}}
 .row{{display:flex;gap:18px;flex-wrap:wrap}} .row>.card{{flex:1;min-width:330px}}
 .note{{color:var(--muted);font-size:13px;margin:0 0 14px;line-height:1.55}}
 footer{{color:var(--muted);font-size:12px;padding:0 32px 30px}}
 @media (max-width:760px){{ header,.kpis,.tabs,.panel,footer{{padding-left:16px;padding-right:16px}} }}
</style></head><body>
<header>
  <h1>Customer Analytics Dashboard</h1>
  <div class="sub">Online Retail II · Digital Assignment 3 · Suyash Agrawal (24BDS1167) · Programming for Data Science</div>
</header>
<div class="kpis">{kpi_html}</div>
<div class="tabs">{tabbar}</div>

<div class="panel on" id="overview">
  <p class="note">Each customer has a churn probability from the tuned Gradient Boosting model (threshold 0.453).
  Revenue at risk is that customer's lifetime spend multiplied by their churn probability.</p>
  <div class="card">{div(f_act)}</div>
  <div class="row"><div class="card">{div(f_risk)}</div><div class="card">{div(f_riskrev)}</div></div>
</div>

<div class="panel" id="models">
  <p class="note">Five churn models compared on the same 1,321-customer test set and the same five cross-validation folds.
  Error bars are bootstrap 95% confidence intervals from 1,000 resamples.</p>
  <div class="card">{div(f_auc)}</div>
  <div class="row"><div class="card">{div(f_box)}</div><div class="card">{div(f_roc)}</div></div>
  <div class="card">{div(t_final)}</div>
  <div class="row"><div class="card">{div(t_pair)}</div><div class="card">{div(t_mcn)}</div></div>
</div>

<div class="panel" id="segments">
  <p class="note">Four K-Means segments on scaled RFM. Bubble size is order frequency; spend is on a log scale
  because the wholesale accounts are orders of magnitude above the rest.</p>
  <div class="card">{div(f_seg)}</div>
  <div class="row"><div class="card">{div(f_segchurn)}</div><div class="card">{div(t_seg)}</div></div>
</div>
```

```
<div class="panel" id="revenue">
  <div class="card">{div(f_rev)}</div>
  <div class="card">{div(f_ord)}</div>
  <div class="card">{div(f_tier)}</div>
</div>

<div class="panel" id="retention">
  <p class="note">The actionable output: valuable customers the model places in the high-risk band, ranked by revenue at risk.</p>
  <div class="card">{div(t_top)}</div>
</div>

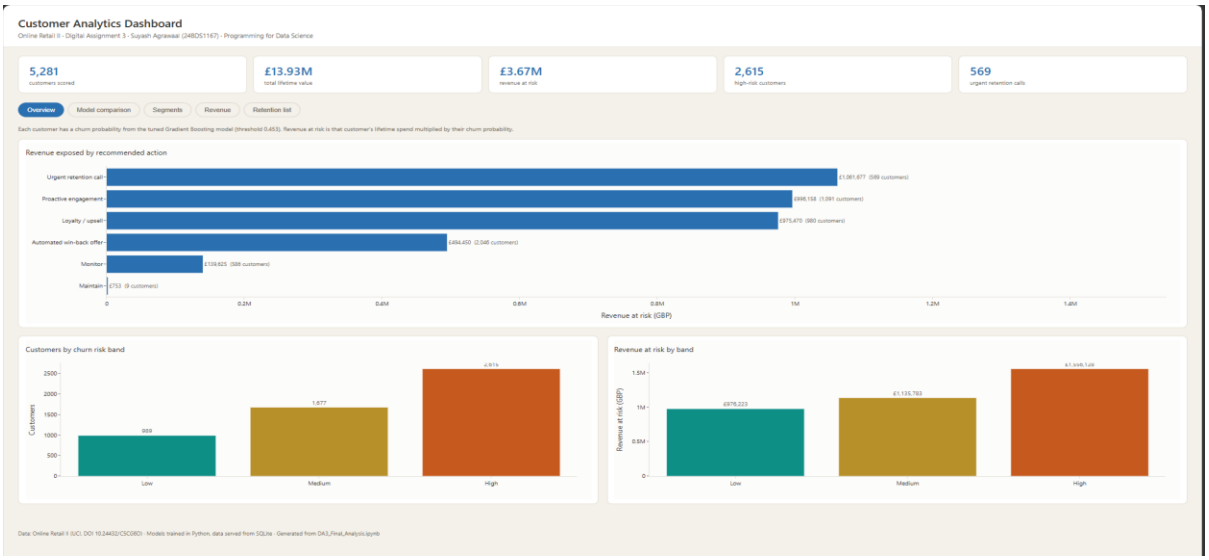
<footer>Data: Online Retail II (UCI, DOI 10.24432/C5CG6D) · Models trained in Python, data served from SQLite · Generated from DA3_Final_Analysis.ipynb</footer>
<script>
 document.querySelectorAll('.tab').forEach(function(b){{
   b.addEventListener('click', function(){{
     document.querySelectorAll('.tab').forEach(function(x){{x.classList.remove('on');}});
     document.querySelectorAll('.panel').forEach(function(p){{p.classList.remove('on');}});
     b.classList.add('on');
     document.getElementById(b.dataset.t).classList.add('on');
     window.dispatchEvent(new Event('resize'));
   }});
 }});
</script></body></html>"""

with open("DA3_Dashboard.html", "w", encoding="utf-8") as f:
    f.write(html)

import os
print("DA3_Dashboard.html written:", round(os.path.getsize("DA3_Dashboard.html")/1e6, 2), "MB")
```

```
DA3_Dashboard.html written: 4.91 MB
```

Output:

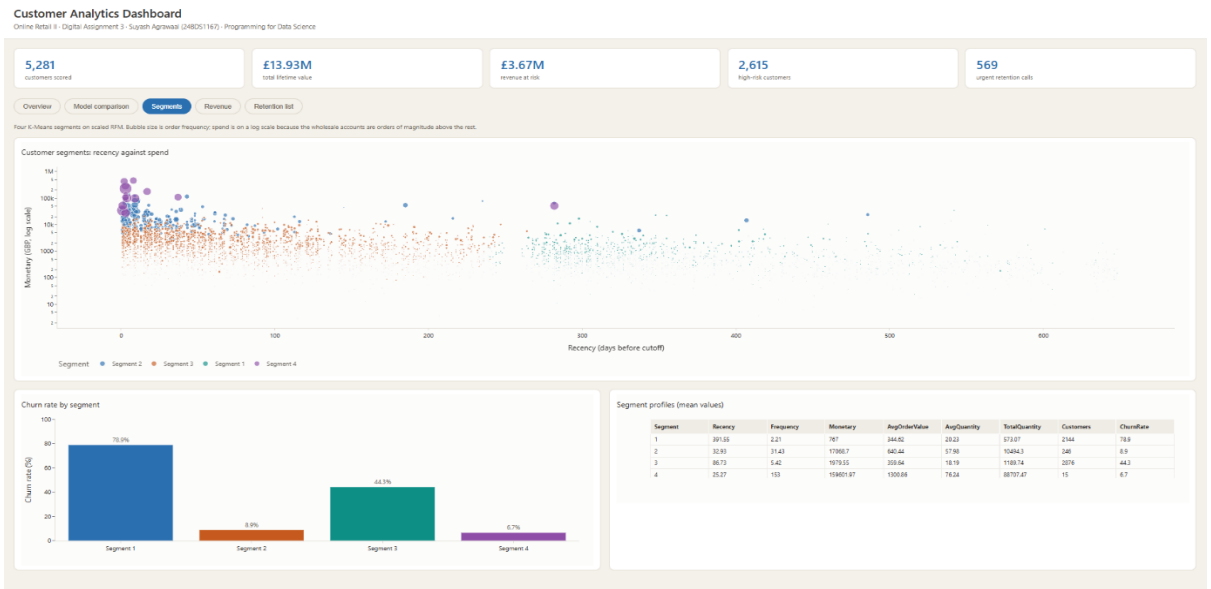


Screenshot 1: *Plotly dashboard, Overview tab*

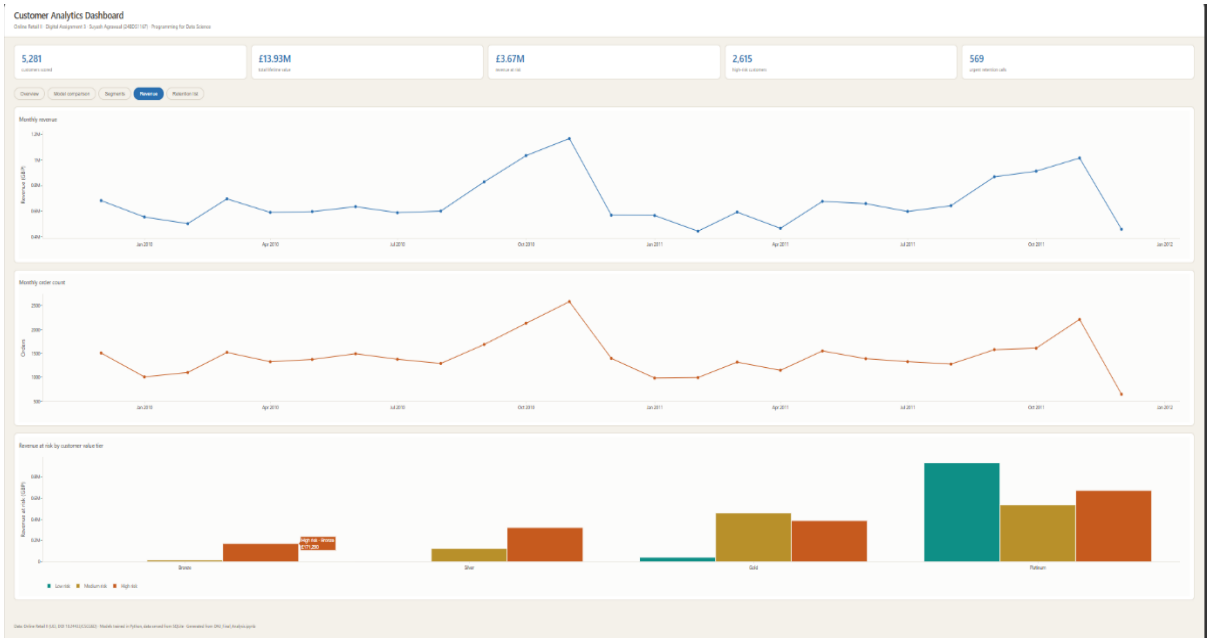


Screenshot 2: *Plotly dashboard, Model comparison tab with the statistical test tables*

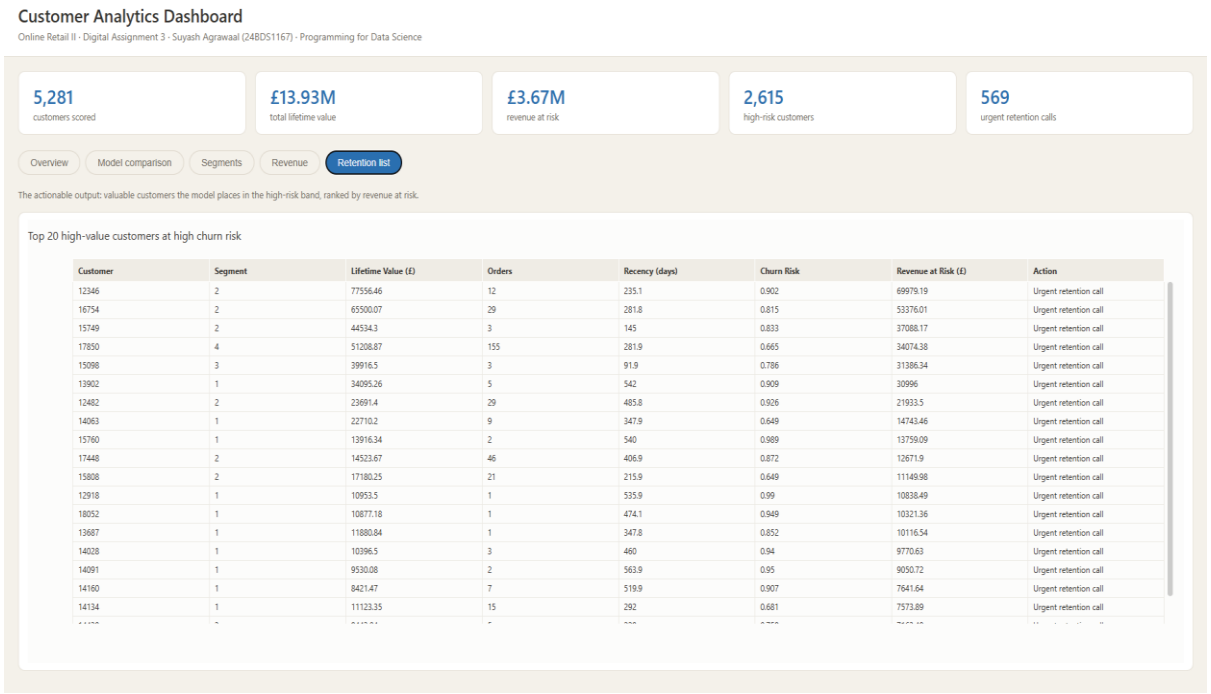
Output:



Screenshot 3: *Plotly dashboard, Segments tab*



Screenshot 4: *Plotly dashboard, Revenue tab*



Screenshot 5: *Plotly dashboard, Retention list tab*

4.3 R Shiny Application

The Shiny application, app.R, adds four filters in a sidebar: churn risk band, customer value tier, segment, and a minimum churn probability slider. Every filter feeds a single reactive expression, so the summary tiles, all five tabs and the downloadable list recalculate together whenever a filter changes.

Element	Behaviour
Summary tiles	Customer count, lifetime value, revenue at risk and average churn risk for the current filter
Overview tab	Revenue by recommended action and customers by risk band, both filtered
Segments tab	Scatter of the filtered customers plus the segment profile table
Models tab	Model comparison chart and the full metric table
Revenue tab	Monthly revenue and revenue at risk by value tier
Customers tab	Sortable and searchable table of every filtered customer
Download button	Exports the current filtered selection as a CSV file

Table 6: *Reactive elements of the Shiny application*

The user interface is defined with page_sidebar from the bslib package, which places the filters on the left and the value boxes and tabs on the right.

Code:

```r
library(shiny); library(plotly); library(DT); library(dplyr); library(readr); library(bslib)

setwd(dirname(rstudioapi::getActiveDocumentContext()$path))   # run from the app folder

scores  <- read_csv("da3_customer_scores.csv",   show_col_types = FALSE)
models  <- read_csv("da3_final_comparison.csv",  show_col_types = FALSE)
segprof <- read_csv("da3_segment_profiles.csv",  show_col_types = FALSE)
monthly <- read_csv("da3_monthly_revenue.csv",   show_col_types = FALSE)
feats   <- read_csv("customer_features.csv",     show_col_types = FALSE)

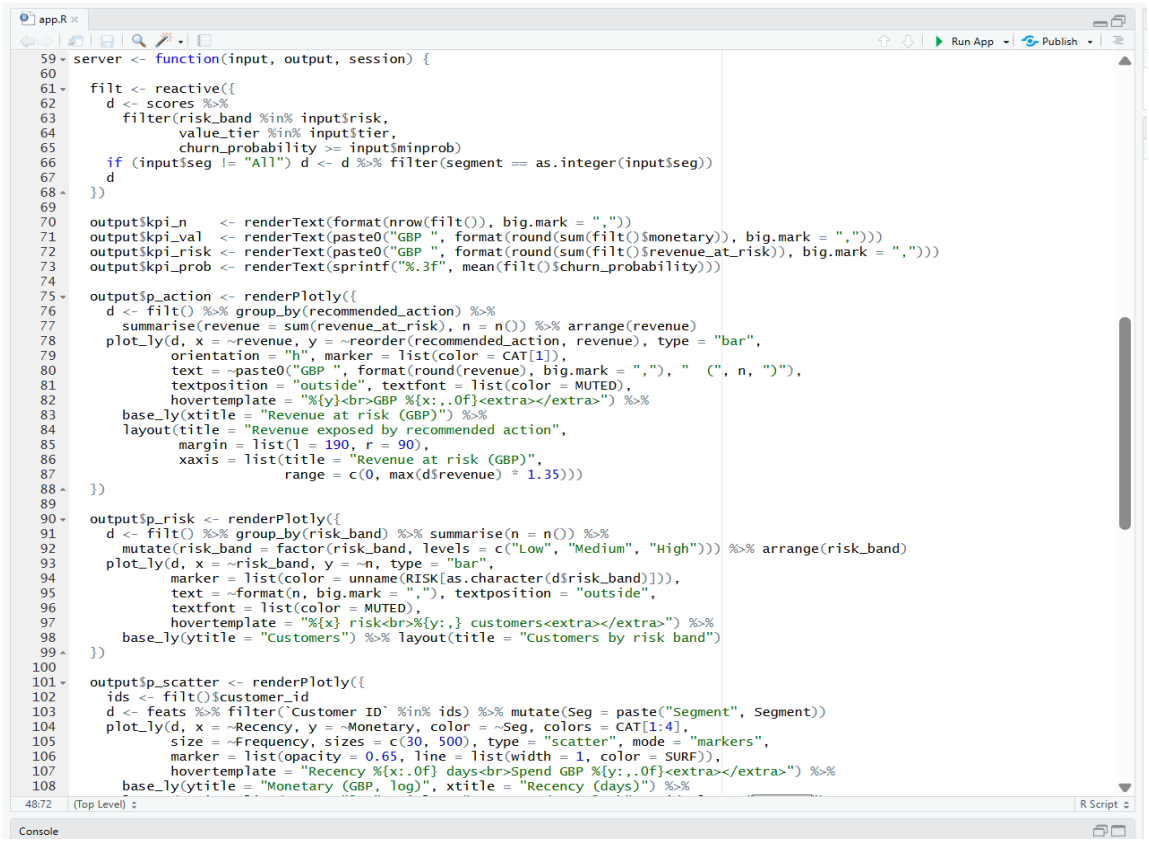
CAT  <- c("#2A6FB0", "#C65A1E", "#0E8F86", "#8E4EA8", "#89902A")
RISK <- c(Low = "#0E8F86", Medium = "#89902A", High = "#C65A1E")
SURF <- "#fcfcfb"; INK <- "#33312e"; MUTED <- "#6f6a62"

base_ly <- function(p, ytitle = "", xtitle = "") {
  p %>% layout(paper_bgcolor = SURF, plot_bgcolor = SURF,
               font = list(family = "Segoe UI, Arial", color = INK, size = 13),
               yaxis = list(title = ytitle, gridcolor = "#eceae4", zeroline = FALSE),
               xaxis = list(title = xtitle, showgrid = FALSE),
               margin = list(l = 70, r = 30, t = 40, b = 60),
               legend = list(orientation = "h", y = -0.2))
}

ui <- page_sidebar(
  title = "Customer Analytics Dashboard - DA3",
  theme = bs_theme(bg = "#f4f1ea", fg = INK, primary = "#2A6FB0", base_font = font_google("Inter")),
  sidebar = sidebar(
    width = 290,
    h5("Filters"),
    checkboxGroupInput("risk", "Churn risk band",
                       choices = c("Low", "Medium", "High"), selected = c("Low", "Medium", "High")),
    checkboxGroupInput("tier", "Customer value tier",
                       choices = c("Bronze", "Silver", "Gold", "Platinum"),
                       selected = c("Bronze", "Silver", "Gold", "Platinum")),
    selectInput("seg", "Segment", choices = c("All", sort(unique(scores$segment))), selected = "All"),
    sliderInput("minprob", "Minimum churn probability", 0, 1, 0, step = 0.05),
    hr(),
    downloadButton("dl", "Download filtered list", class = "btn-primary btn-sm")
  ),
  layout_columns(
    fill = FALSE,
    value_box("Customers", textOutput("kpi_n"), theme = "primary"),
    value_box("Lifetime value", textOutput("kpi_val")),
    value_box("Revenue at risk", textOutput("kpi_risk")),
    value_box("Average churn risk", textOutput("kpi_prob"))
  ),
  navset_card_tab(
    nav_panel("Overview",   plotlyOutput("p_action", height = "380px"),
              plotlyOutput("p_risk",   height = "340px")),
    nav_panel("Segments",   plotlyOutput("p_scatter", height = "480px"),
              DTOutput("t_seg")),
    nav_panel("Models",     plotlyOutput("p_models", height = "400px"), DTOutput("t_models")),
    nav_panel("Revenue",    plotlyOutput("p_month", height = "380px"),
              plotlyOutput("p_tier",  height = "380px")),
    nav_panel("Customers",  DTOutput("t_cust"))
  )
)
```

The server function holds one reactive expression, filt(), that applies all four filters. Every output in the application is derived from it, which is what keeps the tiles, the charts and the table consistent with each other.

Code:



```
server <- function(input, output, session) {

  filt <- reactive({
    d <- scores %>%
      filter(risk_band %in% input$risk,
             value_tier %in% input$tier,
             churn_probability >= input$minprob)
    if (input$seg != "All") d <- d %>% filter(segment == as.integer(input$seg))
    d
  })

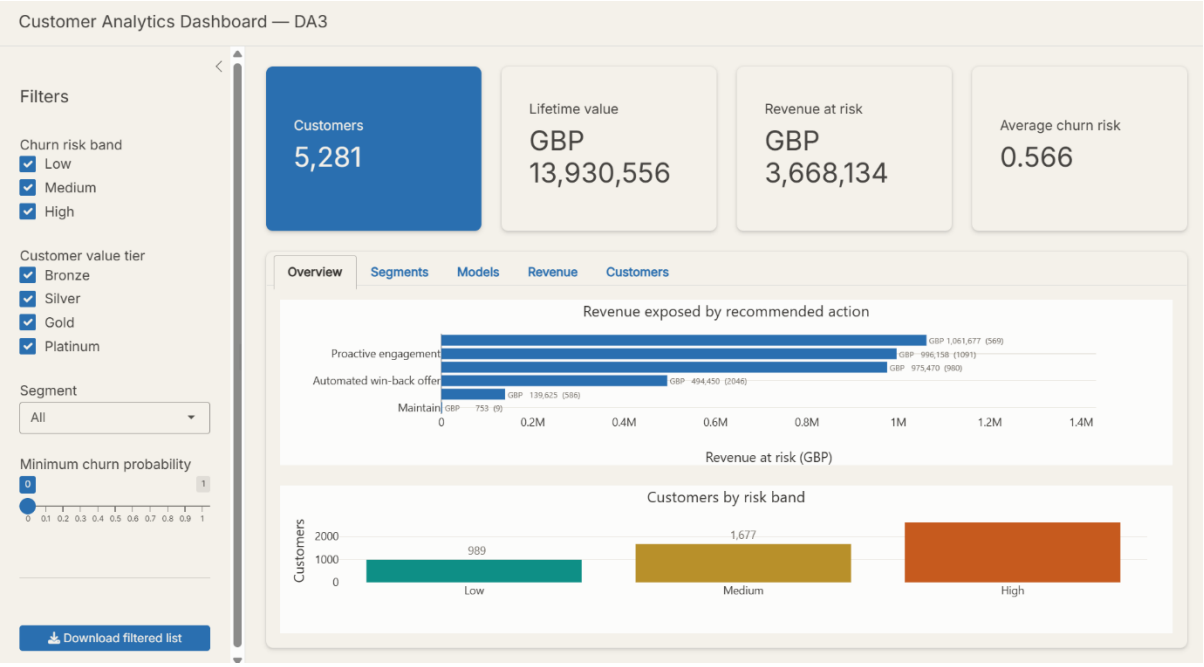
  output$kpi_n    <- renderText(format(nrow(filt()), big.mark = ","))
  output$kpi_val  <- renderText(paste0("GBP ", format(round(sum(filt()$monetary)), big.mark = ",")))
  output$kpi_risk <- renderText(paste0("GBP ", format(round(sum(filt()$revenue_at_risk)), big.mark = ",")))
  output$kpi_prob <- renderText(sprintf("%.3f", mean(filt()$churn_probability)))

  output$p_action <- renderPlotly({
    d <- filt() %>% group_by(recommended_action) %>%
      summarise(revenue = sum(revenue_at_risk), n = n()) %>% arrange(revenue)
    plot_ly(d, x = ~revenue, y = ~reorder(recommended_action, revenue), type = "bar",
            orientation = "h", marker = list(color = CAT[1]),
            text = ~paste0("GBP ", format(round(revenue), big.mark = ","), "  (", n, ")"),
            textposition = "outside", textfont = list(color = MUTED),
            hovertemplate = "%{y}<br>GBP %{x:,.0f}<extra></extra>") %>%
      base_ly(xtitle = "Revenue at risk (GBP)") %>%
      layout(title = "Revenue exposed by recommended action",
             margin = list(l = 190, r = 90),
             xaxis = list(title = "Revenue at risk (GBP)",
                          range = c(0, max(d$revenue) * 1.35)))
  })

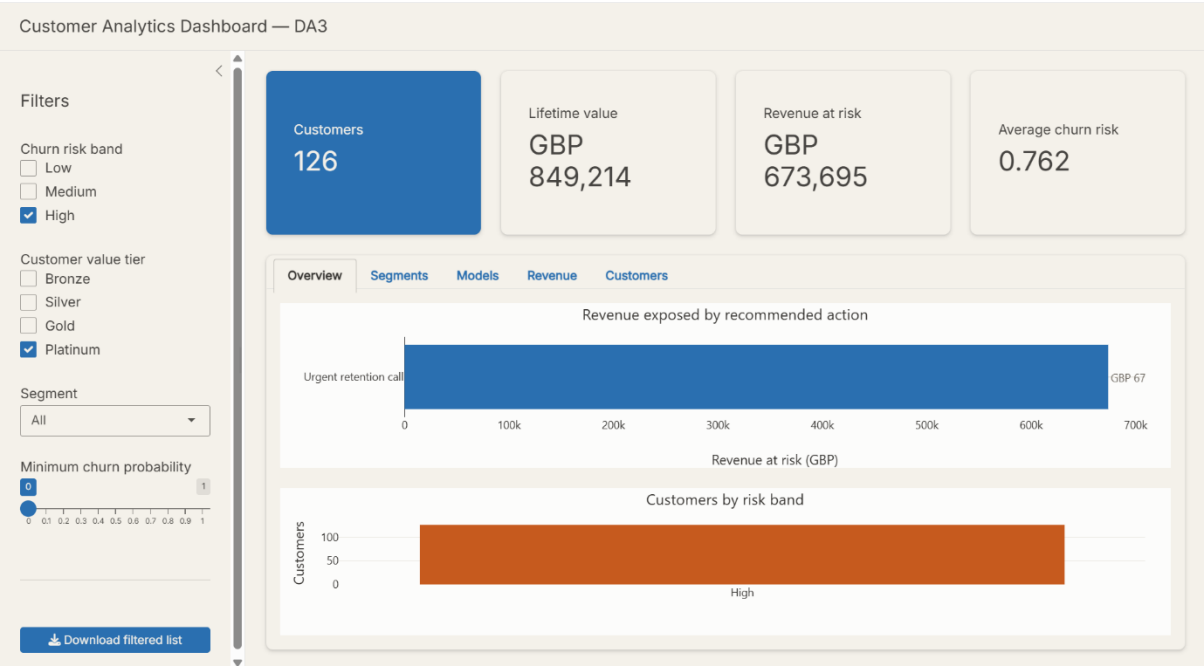
  output$p_risk <- renderPlotly({
    d <- filt() %>% group_by(risk_band) %>% summarise(n = n()) %>%
      mutate(risk_band = factor(risk_band, levels = c("Low", "Medium", "High"))) %>% arrange(risk_band)
    plot_ly(d, x = ~risk_band, y = ~n, type = "bar",
            marker = list(color = unname(RISK[as.character(d$risk_band)])),
            text = ~format(n, big.mark = ","), textposition = "outside",
            textfont = list(color = MUTED),
            hovertemplate = "%{x} risk<br>%{y:,} customers<extra></extra>") %>%
      base_ly(ytitle = "Customers") %>% layout(title = "Customers by risk band")
  })

  output$p_scatter <- renderPlotly({
    ids <- filt()$customer_id
    d <- feats %>% filter(`Customer ID` %in% ids) %>% mutate(Seg = paste("Segment", Segment))
    plot_ly(d, x = ~Recency, y = ~Monetary, color = ~Seg, colors = CAT[1:4],
            size = ~Frequency, sizes = c(30, 500), type = "scatter", mode = "markers",
            marker = list(opacity = 0.65, line = list(width = 1, color = SURF)),
            hovertemplate = "Recency %{x:.0f} days<br>Spend GBP %{y:,.0f}<extra></extra>") %>%
      base_ly(ytitle = "Monetary (GBP, log)", xtitle = "Recency (days)") %>%
      layout(yaxis = list(type = "log", title = "Monetary (GBP, log)", gridcolor = "#eceae4"),
             title = "Segments: recency against spend")
  })
  output$t_seg <- renderDT(datatable(segprof, rownames = FALSE,
                                     options = list(dom = "t", pageLength = 10)))
  output$p_models <- renderPlotly({
    d <- models %>% arrange(`Test ROC-AUC`)
    plot_ly(d, x = ~`Test ROC-AUC`, y = ~reorder(Model, `Test ROC-AUC`), type = "bar",
            orientation = "h", marker = list(color = CAT[1]),
            text = ~sprintf("%.3f", `Test ROC-AUC`), textposition = "outside",
            textfont = list(color = MUTED),
            hovertemplate = "%{y}<br>AUC %{x:.4f}<extra></extra>") %>%
      base_ly(xtitle = "Test ROC-AUC") %>%
      layout(title = "Churn model comparison", xaxis = list(range = c(0.70, 0.86)))
  })
  output$t_models <- renderDT(datatable(models %>% mutate(across(where(is.numeric), ~round(., 4))),
                                        rownames = FALSE, options = list(dom = "t")))
  output$p_month <- renderPlotly({
    plot_ly(monthly, x = ~month, y = ~revenue, type = "scatter", mode = "lines+markers",
            line = list(color = CAT[1], width = 2), marker = list(size = 8),
            hovertemplate = "%{x}<br>GBP %{y:,.0f}<extra></extra>") %>%
      base_ly(ytitle = "Revenue (GBP)") %>% layout(title = "Monthly revenue")
  })
  output$p_tier <- renderPlotly({
    d <- filt() %>% group_by(value_tier, risk_band) %>%
      summarise(revenue = sum(revenue_at_risk), .groups = "drop") %>%
      mutate(value_tier = factor(value_tier, levels = c("Bronze","Silver","Gold","Platinum")))
    plot_ly(d, x = ~value_tier, y = ~revenue, color = ~risk_band,
            colors = RISK, type = "bar",
            hovertemplate = "%{x}<br>GBP %{y:,.0f}<extra></extra>") %>%
      base_ly(ytitle = "Revenue at risk (GBP)") %>%
      layout(barmode = "group", title = "Revenue at risk by value tier")
  })
  output$t_cust <- renderDT({
    datatable(filt() %>%
                select(customer_id, segment, monetary, frequency, recency,
                       churn_probability, risk_band, value_tier,
                       revenue_at_risk, recommended_action) %>%
                arrange(desc(revenue_at_risk)),
              rownames = FALSE, filter = "top",
              options = list(pageLength = 15, scrollX = TRUE)) %>%
      formatRound(c("monetary", "revenue_at_risk"), 2) %>%
      formatRound("churn_probability", 3)
  })
  output$dl <- downloadHandler(
    filename = function() "filtered_customers.csv",
    content  = function(file) write_csv(filt(), file)
  )
}
shinyApp(ui, server)
```

Output:



Screenshot 6: *Shiny application with all filters selected: 5,281 customers, GBP 3,668,134 at risk*

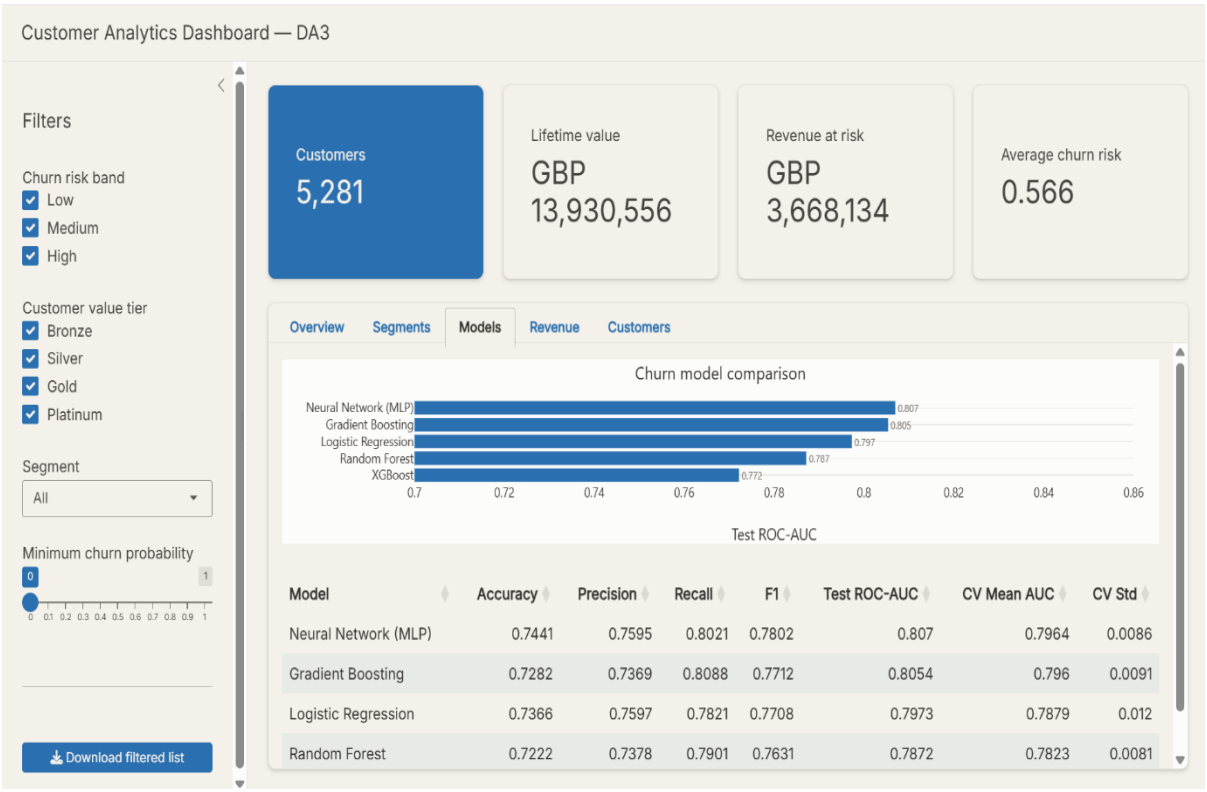


Screenshot 7: *The same view filtered to High risk and Platinum tier: 126 customers, GBP 673,695 at risk*

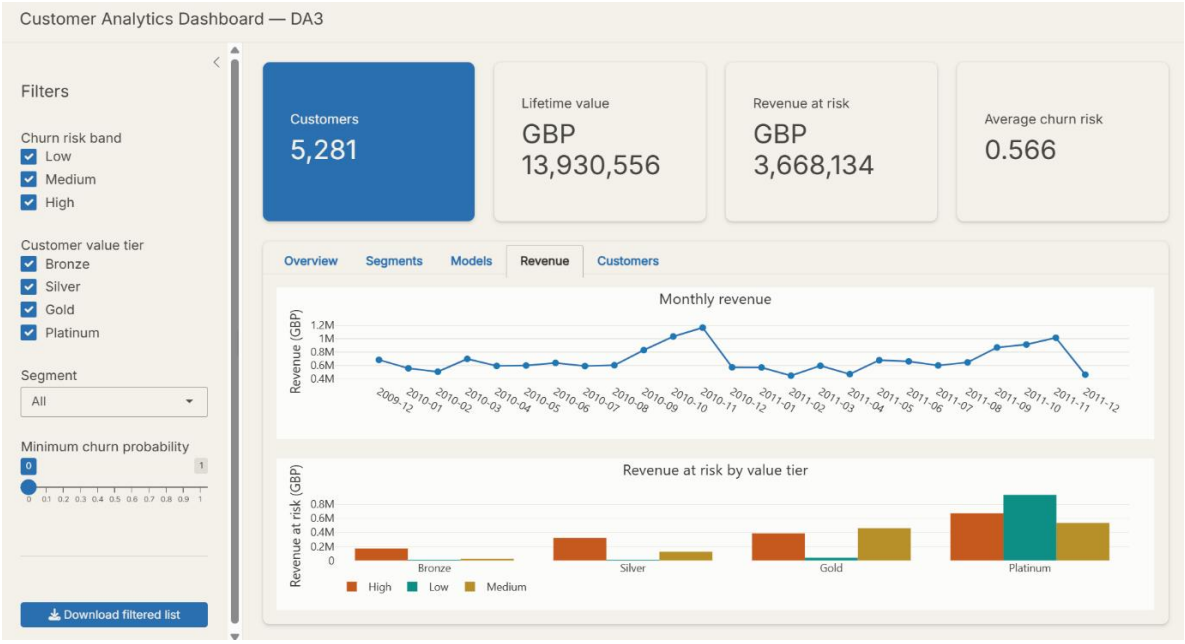
These two screenshots are the clearest demonstration that the application is reactive rather than static. Selecting only the high-risk Platinum customers reduces the population from 5,281 to 126 and raises the average churn risk from 0.566 to 0.762, with every chart redrawn for that subset.



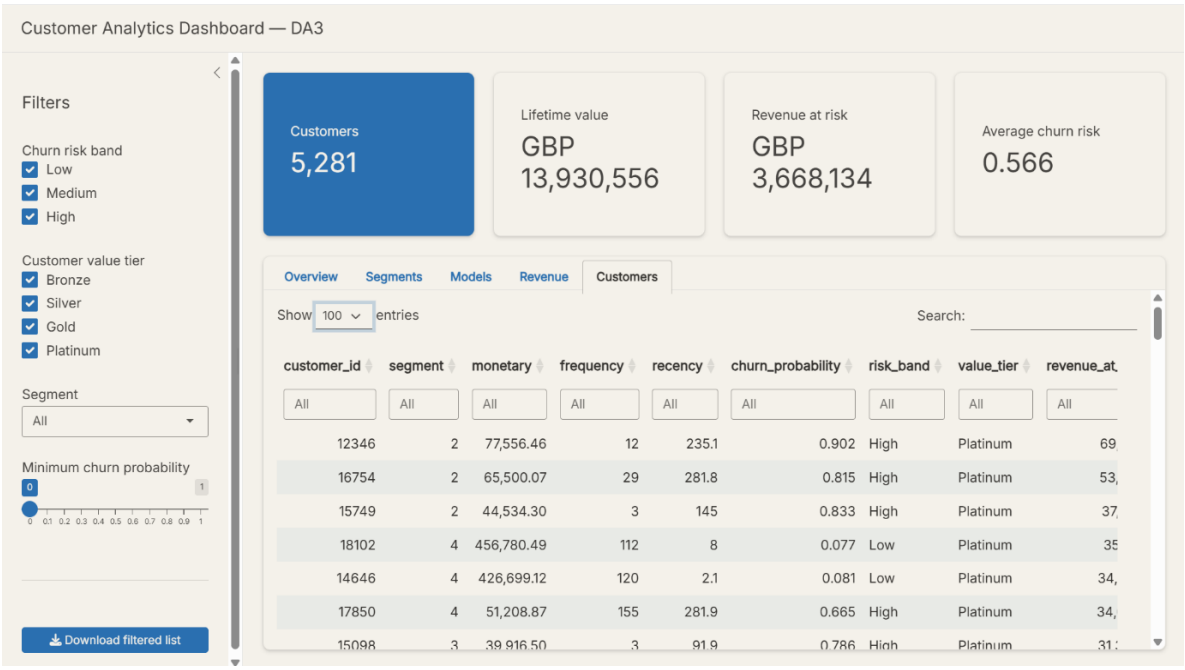
Screenshot 8: *Shiny application, Segments tab*



Screenshot 9: *Shiny application, Models tab*



Screenshot 10: *Shiny application, Revenue tab*



Screenshot 11: *Shiny application, Customers tab with search, sorting and CSV download*

4.4 Comparison of the Two Dashboards

Aspect	Plotly HTML	R Shiny
Runs on	Any browser, offline	R session or a Shiny server
Interactivity	Hover, zoom, tab switching	Full filtering and recomputation
Data handling	Fixed at build time	Recalculated on every filter change
Export	Static file, easy to submit	CSV download of the filtered list
Best suited to	Sharing a fixed set of results	Exploring the customer base

Table 7: *Plotly dashboard compared with the Shiny application*

5. Result Interpretation

5.1 Customer Segments

The four K-Means segments turn out to be recognisable customer types rather than arbitrary groupings, and their churn rates differ sharply.

Segment	Customers	Avg. Recency (days)	Avg. Frequency	Avg. Monetary (GBP)	Churn rate	Description
1	2,144	391.55	2.21	767	78.9%	Dormant, largely lost
2	246	32.93	31.43	17,069	8.9%	Loyal high spenders
3	2,876	86.73	5.42	1,980	44.3%	Active regular buyers
4	15	25.27	153.00	159,602	6.7%	Elite wholesale accounts

Table 8: *Segment profiles with churn rate*

The churn rate falls from 78.9% in Segment 1 to 6.7% in Segment 4, and it falls in step with recency and frequency. Segment 1 customers last purchased more than a year before the cutoff and ordered barely twice, so most of them never return. Segment 4 contains fifteen wholesale accounts averaging GBP 159,602 of spend across 153 orders each, and almost all of them stay.

This gives the clustering practical meaning. The segments were built from RFM values without knowing anything about churn, yet they separate churn behaviour cleanly, which is independent evidence that RFM captures something real about these customers.

5.2 Churn Risk and Revenue Exposure

Risk band	Customers	Avg. churn probability	Lifetime value (GBP)	Revenue at risk (GBP)
Low (below 0.3)	989	0.154	9,371,701	976,223
Medium (0.3 to 0.6)	1,677	0.465	2,539,303	1,135,783
High (above 0.6)	2,615	0.787	2,019,552	1,556,128
Total	5,281	0.566	13,930,556	3,668,134

Table 9: *Customers, lifetime value and revenue at risk by band*

The two value columns point in opposite directions and that is the central business insight of the project. Most of the money, GBP 9.37 million, sits with the 989 low-risk customers, who are not going anywhere. The high-risk band holds far less lifetime value, GBP 2.02 million, but because those customers are likely to leave, their probability-weighted exposure of GBP 1.56 million is the largest of the three bands.

Across all bands the total exposure is GBP 3.67 million, which is 26% of the historical customer value in the database. This is a measure of exposure based on past spend, not a forecast of future revenue.

5.3 Behavioural Drivers of Churn

The explainability work in DA2 identified Recency as the strongest predictor in both tree ensembles, followed by Frequency and the quantity measures, with the country indicators contributing very little. The SHAP analysis added direction: high Recency pushes a prediction towards churn, while high TotalQuantity and Monetary push it away.

The segment table in Section 5.1 shows the same relationship from a completely different angle. Average recency rises from 25 days in Segment 4 to 392 days in Segment 1, and churn rate rises with it from 6.7% to 78.9%. Two independent methods agreeing on the same driver is stronger evidence than either on its own.

5.4 Business Reading of the Results

- Act on exposure, not probability. The highest-probability customers are mostly low-value; the 569 customers in the urgent band carry GBP 1.06 million of exposure between them and deserve individual attention.
- Match the channel to the value. An automated offer is proportionate for the 2,046 low-value high-risk customers; a phone call is not.
- Protect Segment 2 and Segment 4. Together they are 261 customers holding roughly GBP 6.6 million of lifetime value, and their churn rates are already low, so the goal there is to avoid losing them rather than to win them back.
- Treat Segment 1 as a win-back problem, not a retention problem. At 78.9% churn and GBP 767 of average spend, these customers have effectively already left.

5.5 Limits of the Interpretation

Three cautions apply to everything above. The churn label is a 90-day definition, so a customer who returns in month four is counted as churned. Revenue at risk uses historical spend as a proxy for future value, which understates growing customers and overstates declining ones. And the action matrix is a reasonable business rule, not an optimised policy, because the dataset contains no record of what retention offers cost or how often they succeed.

6. Conclusion, Limitations and Future Scope

6.1 Conclusion

Across three phases this project took a file of 1,067,371 raw transaction rows and ended with a ranked, actionable retention list stored in a queryable database. DA1 cleaned the data to 779,425 transactions. DA2 converted those into 5,281 customer records using RFM features, defined three prediction targets, built a normalized SQLite database shared between R and Python, and trained twenty-four models. DA3 tested the models properly, combined their outputs into one scoring view and presented the whole thing through two dashboards.

Four findings stand out. First, the features set the ceiling, not the algorithm: five very different models land between 0.7722 and 0.8070 ROC-AUC, and the top two are statistically indistinguishable. Second, defining the label honestly mattered more than any modelling choice; the 90-day time split replaced an implausible 100% accuracy with a defensible 70 to 74%. Third, cheap adjustments beat expensive ones, since moving the decision threshold raised recall by 0.064 while a full grid search gained 0.002 of ROC-AUC. Fourth, value and risk must be combined, because the highest-risk customers are not the ones holding the most money.

The final output is concrete: 5,281 customers scored, GBP 3.67 million of revenue exposure quantified, and 569 customers named for immediate retention effort.

6.2 Limitations

- Churn is defined by a single 90-day window. A longer or shorter window would change both the labels and the reported accuracy.
- Predictive performance plateaus near 0.80 ROC-AUC. The dataset carries no marketing, web-session, complaint or demographic data, so the behavioural features alone set the limit.
- Segment 4 contains only fifteen customers, which makes any statistic about that segment fragile.
- The segment classification task reaches about 0.99 accuracy because it relearns a boundary K-Means already drew from the same features; it is not independent validation of the clustering.
- Hyperparameter tuning was applied to the churn task only. The high-value and segment models still use default settings.

- Some probability calibration curves deviate from the diagonal, so the raw probabilities should be calibrated before being used for budget calculations.
- Revenue at risk is based on historical spend, not on predicted future value, and no cost of intervention is modelled.
- The data is two years old and from a single UK retailer, so the specific patterns should not be assumed to generalise to other businesses.

6.3 Future Scope

- Add time-dependent features such as inter-purchase gaps, trend in spend over the last three months, and seasonality, which are the most likely source of improvement beyond 0.80.
- Replace the fixed 90-day label with a survival model that predicts when a customer is likely to lapse rather than whether they lapse within a fixed window.
- Calibrate the probabilities with Platt scaling or isotonic regression so that the risk bands correspond to true probabilities.
- Extend tuning and SMOTE experiments to the high-value and segment tasks.
- Introduce a cost matrix with the real cost of a retention offer and the real margin on a retained customer, then optimise the threshold for profit rather than F1.
- Deploy the Shiny application to shinyapps.io and schedule the scoring pipeline so that the scores refresh as new transactions arrive.
- Validate the clustering independently, for example with silhouette analysis across different values of k, and test whether a different k separates the dormant group more finely.

8. References

[1] Chang, W., Cheng, J., Allaire, J. J., Sievert, C., et al. (2024). *shiny: Web Application Framework for R.* https://shiny.posit.co

[2] Chen, D. (2012). *Online Retail II* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CG6D

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

[4] Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7, 1–30.

[5] Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7), 1895–1923. https://doi.org/10.1162/089976698300017197

[6] Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman and Hall/CRC.

[7] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

[8] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*, 4765–4774.

[9] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, 56–61. https://doi.org/10.25080/Majora-92bf1922-00a

[10] McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157. https://doi.org/10.1007/BF02295996

[11] Nadeau, C., & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning*, 52(3), 239–281. https://doi.org/10.1023/A:1024068626366

[12] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[13] Plotly Technologies Inc. (2015). *Collaborative data science*. Montreal: Plotly Technologies Inc. https://plot.ly

[14] Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, 92–96. https://doi.org/10.25080/Majora-92bf1922-011

[15] Sievert, C. (2020). *Interactive Web-Based Data Visualization with R, plotly, and shiny*. Chapman and Hall/CRC.

[16] Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

[17] Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83. https://doi.org/10.2307/3001968

[18] Xiahou, X., & Harada, Y. (2022). B2C e-commerce customer churn prediction based on K-Means and SVM. *Journal of Theoretical and Applied Electronic Commerce Research*, 17(2), 458–475. https://doi.org/10.3390/jtaer17020024